Pick-and-Place Robot with ROS2 and Arduino for Color-Based Sorting

Samriddhi Vikram Kafle
kafle.sv@edu.rptu.de
RPTU Kaiserslautern-Landau

Sara Bavi
gux89nar@rptu.de
RPTU Kaiserslautern-Landau

Joel Agustin Sanchez
zij22syh@rptu.de
RPTU Kaiserslautern-Landau

Herine Atieno Omondi
lek33ril@rptu.de
RPTU Kaiserslautern-Landau

***Abstract*—This report presents the development of a Pick-and-Place robotic system using an Arduino Braccio robotic arm with an Arduino Mega 2560. The system is designed to autonomously identify, pick up, and sort objects based on their color. The robot is equipped with a camera mounted in a fixed position to capture the object's color, then perform sorting accordingly. The system is built using ROS2, RViz, URDF, and MoveIt to control the robot's motion. Custom ROS2 nodes were created to manage tasks like object detection, motion planning, and sorting. Data is transferred to the robot using the Action interface, ensuring smooth communication during the task execution.**

***Index Terms*—Pick-and-Place, Arduino Braccio, ROS2 Humble, Color Detection, MoveIt, Serial Communication**

I. Introduction

In recent years, the demand for autonomous robotic systems has increased across industries, particularly in manufacturing and logistics, where pick-and-place operations are common. Robots that can detect, identify, and sort objects autonomously contribute to increased efficiency and reduced manual labor. Inspired by this trend, the goal of this project was to design a compact, low-cost Pick-and-Place robotic system suitable for educational and research environments.

The motivation behind this project was to explore how widely accessible platforms—such as the Arduino Braccio arm and open-source software like ROS2—can be integrated to create a functional robotic application. By using color detection as the sorting criteria, the system simulates basic industrial automation tasks and serves as a valuable learning tool for robotic control, motion planning, and perception.

The objective of this project is to design and implement an automated Pick-and-Place system that sorts objects based on their color. The Arduino Braccio robotic arm with an Arduino Mega 2560 is utilized to perform the object manipulation tasks. The robot interacts with a camera system for color detection, and based on the result, sorts objects into predefined places. The project integrates software and hardware components such as ROS2 tools, MoveIt, microcontroller, and servo motors to enable precise control of the robot's movements and actions.

This report outlines the steps involved in the design, implementation, and testing of the system, focusing on the use of ROS2 Humble for communication and control, and MoveIt for motion planning.

Fig. 1. Arduino Braccio manipulator

II. System Design

A. Hardware Components

- **Arduino Braccio Robotic Arm:** 6-DOF robotic manipulator for pick-and-place tasks.
- **Arduino Mega 2560:** Microcontroller board used to control the Braccio arm.
- **Camera:** USB webcam with 8.0-megapixel resolution used for object detection and color recognition.
- **Computer/PC running ROS2 (OS Linux):** Handles image processing, motion planning, and robot control.
- **Colored Objects:** Items used for the sorting task.

B. Software Components

The software architecture of the project is based on the Robot Operating System 2 (ROS 2) and additional open-source tools. The main components include:

- **ROS 2 Humble**: A middleware framework used for modular robot control, inter-node communication, and system integration.
- **Ubuntu 22.04**: The operating system used to run ROS 2 and all required packages.

- **RViz**: A 3D visualization tool used for visualizing the robot model, planning scene, and motion trajectories.
- **MoveIt 2**: A motion planning library integrated with ROS 2, used for planning and executing robot arm trajectories.
- **OpenCV**: An open-source computer vision library used for image processing tasks, including object detection and color recognition.
- **Arduino IDE**: A software platform used to program the Arduino Mega 2560 board for controlling the Braccio robotic arm.
- **Serial Communication Interface**: Enables communication between ROS 2 and the Arduino board for sending control commands.

C. Workflow

The workflow of the system is composed of the following sequential steps:

1) The robot moves to its predefined **home position.**
2) The robot picks up the object from a specific location.
3) The object is placed in front of the camera for color detection.
4) The camera detects the color of the object.
5) Based on the detected color, the object is sorted and placed in the corresponding designated area.
6) The robot returns to the home position and prepares for the next cycle.

III. Implementation

A. System Setup and Modelling

The system was implemented using ROS 2 Humble, installed on Windows 11 through the WSL2 (Windows Subsystem for Linux 2) virtual environment. The robotic arm was modeled and configured using the following key components:

- **URDF (Unified Robot Description Format):** It is an XML file that defines what the robot physically looks like and how it can move.
- **SRDF (Semantic Robot Description Format):** It is an XML file that defines how to group and use parts of the robot for motion planning.
- **RVIZ:** A 3D visualization and interaction tool used to simulate and monitor the robot's behavior.
- **MoveIt:** It is the motion planning environment within ROS 2 frame work with functionalities such as trajectory generation, inverse kinematics (IK) and collision avoidance.

B. Motion Planning

The trajectory is calculated for the robot arm to move from the initial position to the object location without any collision. Several ROS nodes were created as detailed below:

- **Motion Planning Nodes:** Those nodes utilize MoveIt to plan and execute the motion of the robot's arm. It ensures that the robot moves in a smooth, collision-free manner.
- **Arduino Communication Node:** This node sends the target joint angles to the Arduino via a serial interface using a predefined communication protocol.

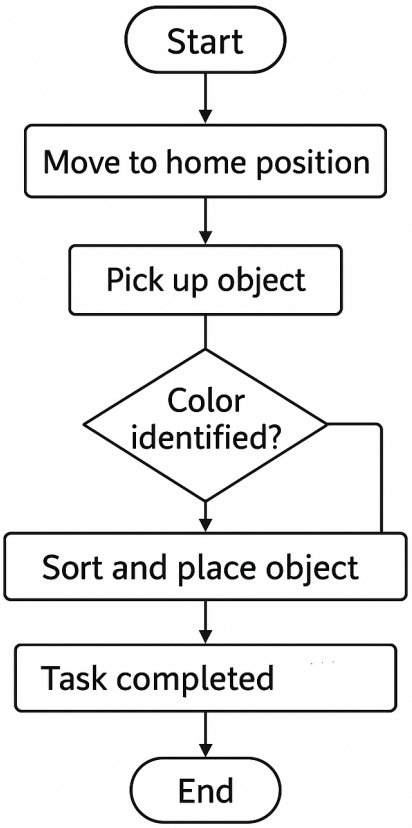

Fig. 2. Summary of the system workflow for object detection and sorting.

C. Object Detection

To accomplish the pick and place task, the robot detects the color of the object by using a camera fixed at a specific position and sorting is done based on the detected color. The detection process involves:

- Image acquisition
- Preprocessing (such as filtering and thresholding)
- Color classification (based on predefined color ranges)

The processed color data is then used to guide the robot's actions. The following nodes are used:

- **Object Detection Node:** This node handles the image capture from the camera, processes the image, and extracts the color information which is then published to a ROS 2 topic /color-detected. OpenCV was used for image processing.
- **Sorting Node:** This node subscribes to the /color-detected topic, then classifies the objects based on the color detected by the camera. It then sends a signal to the motion planning node to place the object in the correct location.

D. Use of Action Interface

The Action interface in ROS2 was utilized for the communication between nodes during task execution. It allows the system to send and receive data asynchronously, ensuring that

the robot's tasks are performed in real-time. For example, once an object is detected, the Action interface communicates the color data to the sorting node, which then triggers the motion planning node to execute the robot's arm movement.

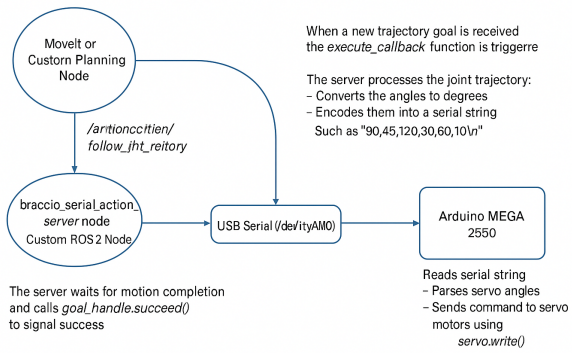

Fig. 3. System architecture: Communication between ROS 2 nodes and Arduino Braccio robot using the action interface and serial connection.

IV. Results

A. Testing and Validation

The system was tested in a controlled environment with a variety of colored objects. The robot successfully identified and sorted objects based on color in real-time. The robot's performance was evaluated based on accuracy (correctly identifying the color) and efficiency (time taken to pick, identify, and place an object).

B. Challenges

Several challenges were encountered during the development process:

- **Setup and Hardware Integration:** Following the installation of ROS2 Humble on windows using WSL2, there were limitations in terms of hardware integration, for instance camera access. A solution was to run ROS2 entirely on Linux to allow for real-time capabilities and seamless hardware integration.
- **Camera Calibration:** Ensuring that the camera consistently detected colors under different lighting conditions was challenging.
- **Motion Planning:** The robot had to be programmed to avoid obstacles and ensure smooth motion during the pick-and-place task.
- **Real-Time Processing:** The system had to be optimized to process the image and execute actions in real-time without delay.
- **Communication with Arduino Braccio Arm:** Several attempts as highlighted below were made to enable communication between MoveIt pipeline and Arduino Braccio Arm:
  - **Rosserial:** This option didn't work since some of the packages required catkin, a build system for ROS1 whereas ROS2 uses colcon, leading to inconsistent behavior during the setup.
  - **Micro-ROS:** A promising alternative however with complex setup and moreover Arduino firmware does not natively support it.
  - **Custom Serial Interface:** Serial communication using a lightweight protocol to interpret joint angles and command servos on Arduino.

These challenges were addressed by adjusting the camera's settings, fine-tuning the motion planning algorithms, and optimizing the ROS2 nodes for faster data processing.

V. Conclusion

This project demonstrated the successful development of a low-cost, autonomous Pick-and-Place system using an Arduino Braccio robotic arm and ROS2 Humble. The system was capable of detecting object colors via a fixed camera and sorting them accordingly through planned robotic arm motions. The integration of key tools such as MoveIt, RViz, and OpenCV enabled a complete workflow from perception to actuation.

Beyond its technical accomplishments, the project serves as a practical example of how open-source tools and accessible hardware can be combined to create useful robotic applications. It offered hands-on experience in motion planning, real-time communication, and sensor integration, which are essential concepts in robotics.

While several challenges—such as hardware integration in WSL2 and consistent camera calibration—were encountered, they provided valuable insights and led to a deeper understanding of real-world system constraints. The final system not only met the initial goals but also laid the foundation for further exploration in the areas of adaptive perception, handling varied object types, and expanding robot autonomy.

Future work may involve improving the object detection pipeline using machine learning, integrating depth sensors for enhanced perception, and expanding the system's ability to operate in dynamic environments. This project illustrates the potential of small-scale robotic systems as both educational platforms and prototypes for larger industrial solutions.

References

[1] S. Macenski, F. Martin, R. White, and J. Ginés, "The Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[2] D. Coleman, M. Söllner, B. Gerkey, S. Jones, and S. Chitta, "Reducing the barrier to entry of complex robotic software: a MoveIt! case study," *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014.

[3] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[4] ROS Wiki Contributors, "URDF - ROS Wiki," Online: http://wiki.ros.org/urdf, accessed Apr. 30, 2025.

[5] Arduino, "Arduino Braccio and Mega 2560 Documentation," Online: https://docs.arduino.cc, accessed Apr. 30, 2025.

[6] Microsoft, "Run ROS on WSL2," Online: https://learn.microsoft.com/en-us/windows/wsl/tutorials/wsl-ros, accessed Apr. 30, 2025.